PYTHON PROGRAMMING

Practical Record

Algorithms • Procedures • Pseudocode • Source Code • Corrected Flowcharts • Sample Output Screenshots

Subject	Python Programming
Record contents	10 basic Python programs
Prepared as	Practical / Lab Record

Note: Output screenshots use sample input values to demonstrate each program. Flowcharts have been revised to show the correct decisions, branches and loop paths.

Contents

1. Student Marks and Pass/Fail
2. Electricity Bill Calculation
3. Gross and Net Salary
4. Banking Menu (Deposit / Withdraw / Balance)
5. Product Billing with Discount
6. Class Attendance Eligibility
7. Password Length Validation
8. Highest, Lowest and Average Marks
9. Bus Passenger Capacity
10. Number Pattern

Program 1: Student Marks and Pass/Fail

Aim: To calculate the total and average marks of a student and decide whether the student passes.

Procedure

1. Read the student's name and marks in three subjects.
2. Add the three marks to calculate the total.
3. Divide the total by 3 to calculate the average.
4. If the average is at least 40, display Pass; otherwise, display Fail.

Algorithm

1. Start.
2. Read the student's name and marks in three subjects.
3. Add the three marks to calculate the total.
4. Divide the total by 3 to calculate the average.
5. If the average is at least 40, display Pass; otherwise, display Fail.
6. Stop.

Pseudocode

```
START
INPUT name, m1, m2, m3
total ← m1 + m2 + m3
average ← total / 3
IF average ≥ 40 THEN DISPLAY Pass
ELSE DISPLAY Fail
END IF
STOP
```

Python Source Code

```
name = input("Enter student name: ")
m1 = int(input("Enter marks in subject 1: "))
m2 = int(input("Enter marks in subject 2: "))
m3 = int(input("Enter marks in subject 3: "))

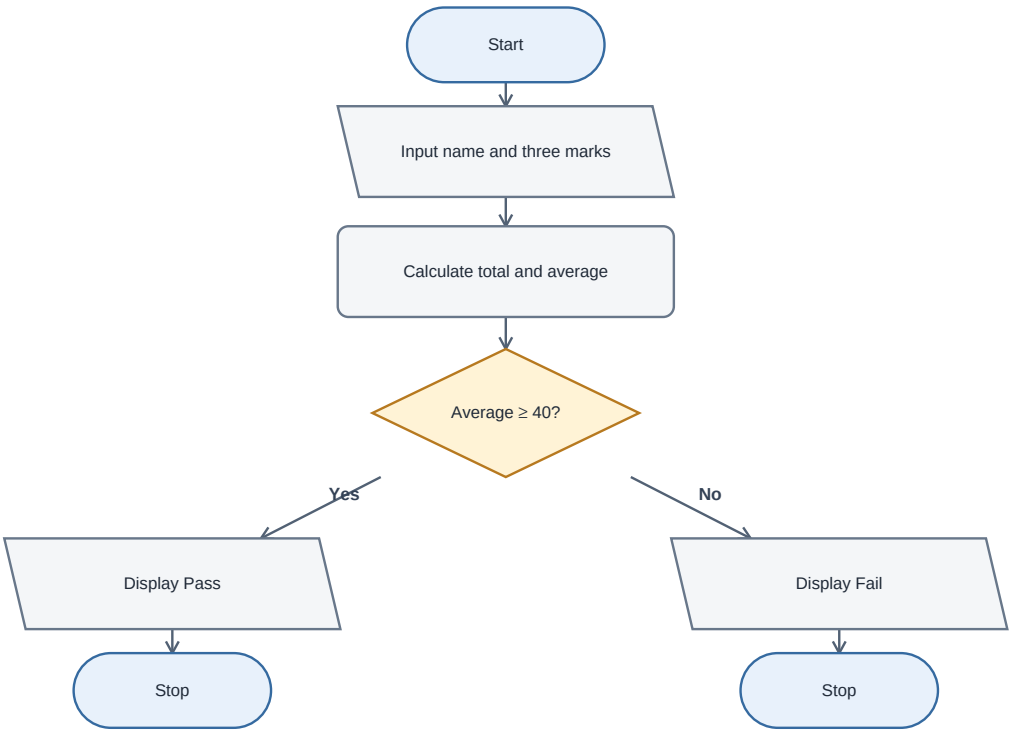
total = m1 + m2 + m3
average = total / 3

print("Student:", name)
print("Total:", total)
print("Average:", average)

if average >= 40:
    print("Pass")
else:
    print("Fail")
```

Program 1: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 1 – Sample Run

Enter student name: Asha
Enter marks in subject 1: 75
Enter marks in subject 2: 68
Enter marks in subject 3: 80
Student: Asha
Total: 223
Average: 74.33333333333333
Pass
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 2: Electricity Bill Calculation

Aim: To calculate an electricity bill using slab-wise unit rates.

Procedure

1. Read the number of electricity units consumed.
2. For up to 100 units, charge ■2 per unit.
3. For 101–200 units, charge ■2 for the first 100 units and ■3 for each additional unit.
4. Above 200 units, add ■5 per unit for consumption beyond 200.
5. Display the bill amount.

Algorithm

1. Start.
2. Read the number of electricity units consumed.
3. For up to 100 units, charge ■2 per unit.
4. For 101–200 units, charge ■2 for the first 100 units and ■3 for each additional unit.
5. Above 200 units, add ■5 per unit for consumption beyond 200.
6. Display the bill amount.
7. Stop.

Pseudocode

```
START
INPUT units
IF units ≤ 100: bill ← units × 2
ELSE IF units ≤ 200: bill ← 100 × 2 + (units − 100) × 3
ELSE: bill ← 100 × 2 + 100 × 3 + (units − 200) × 5
DISPLAY bill
STOP
```

Python Source Code

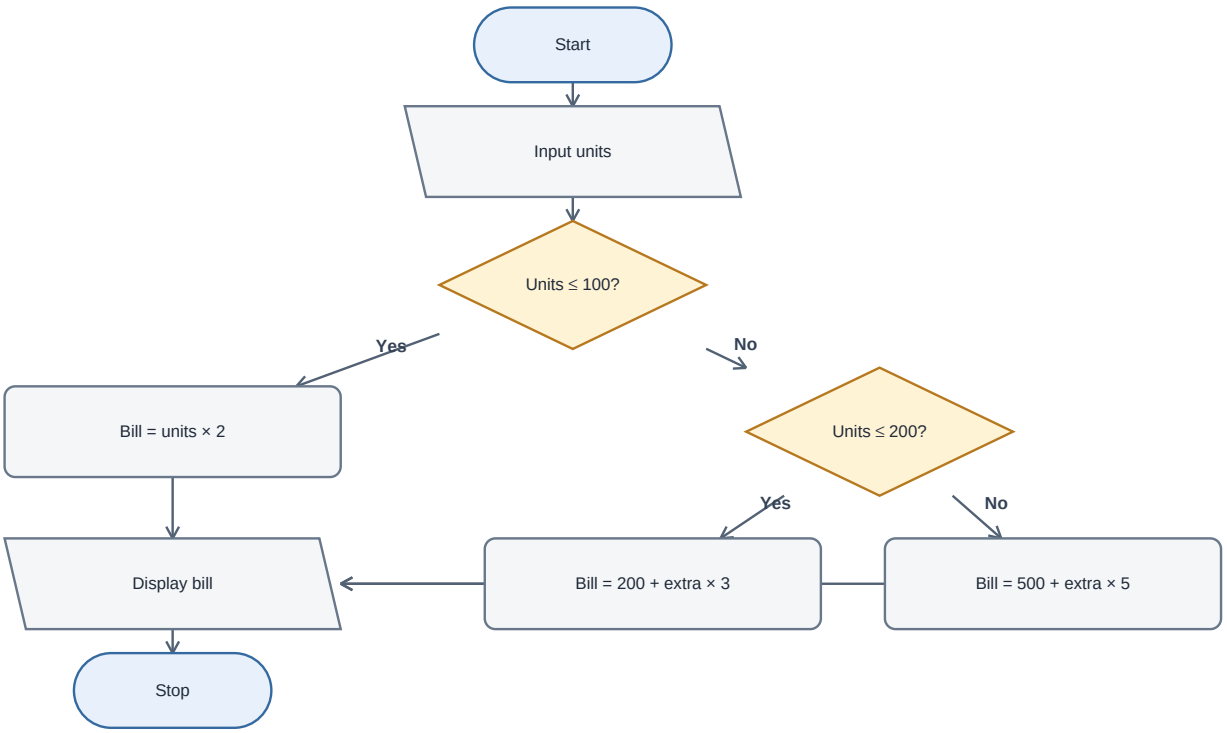
```
units = int(input("Enter electricity units: "))

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

print("Electricity Bill:", bill)
```

Program 2: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 2 – Sample Run

Enter electricity units: 250
Electricity Bill: 650
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 3: Gross and Net Salary

Aim: To calculate HRA, DA, gross salary, deduction and net salary from basic salary.

Procedure

1. Read the basic salary.
2. Calculate HRA as 20% of basic salary.
3. Calculate DA as 10% of basic salary.
4. Add basic salary, HRA and DA to get gross salary.
5. Calculate deduction as 5% of gross salary.
6. Subtract the deduction from gross salary and display the results.

Algorithm

1. Start.
2. Read the basic salary.
3. Calculate HRA as 20% of basic salary.
4. Calculate DA as 10% of basic salary.
5. Add basic salary, HRA and DA to get gross salary.
6. Calculate deduction as 5% of gross salary.
7. Subtract the deduction from gross salary and display the results.
8. Stop.

Pseudocode

```
START
INPUT basic
hra ← basic × 0.20
da ← basic × 0.10
gross ← basic + hra + da
deduction ← gross × 0.05
net ← gross − deduction
DISPLAY gross, net
STOP
```

Python Source Code

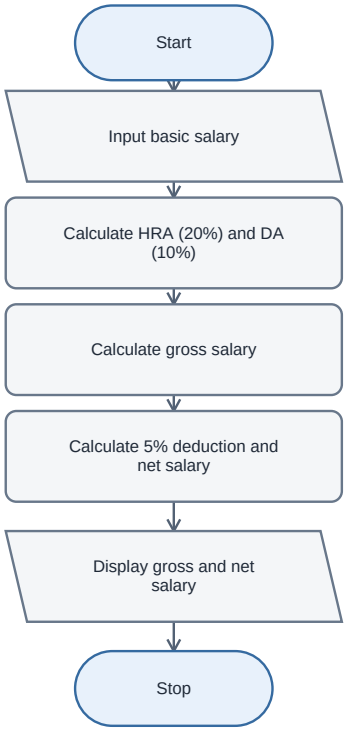
```
basic = float(input("Enter basic salary: "))

hra = basic * 0.20
da = basic * 0.10
gross = basic + hra + da
deduction = gross * 0.05
net = gross - deduction

print("HRA:", hra)
print("DA:", da)
print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net)
```

Program 3: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 3 – Sample Run

Enter basic salary: 20000
HRA: 4000.0
DA: 2000.0
Gross Salary: 26000.0
Deduction: 1300.0
Net Salary: 24700.0
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 4: Banking Menu (Deposit / Withdraw / Balance)

Aim: To create a simple banking menu that allows deposits, withdrawals and balance enquiries.

Procedure

1. Initialize the balance to ■5,000.
2. Display the menu repeatedly until Exit is selected.
3. For Deposit, read an amount and add it to the balance.
4. For Withdraw, check whether sufficient balance is available; if yes, subtract the amount.
5. For Check Balance, display the current balance.
6. For any other menu choice, display Invalid Choice.

Algorithm

1. Start.
2. Initialize the balance to ■5,000.
3. Display the menu repeatedly until Exit is selected.
4. For Deposit, read an amount and add it to the balance.
5. For Withdraw, check whether sufficient balance is available; if yes, subtract the amount.
6. For Check Balance, display the current balance.
7. For any other menu choice, display Invalid Choice.
8. Stop.

Pseudocode

```
START
balance ← 5000
REPEAT
  DISPLAY menu: Deposit, Withdraw, Check Balance, Exit
  INPUT choice
  IF choice = 1: INPUT amount; balance ← balance + amount
  ELSE IF choice = 2: INPUT amount; IF amount ≤ balance subtract; ELSE show Insufficient Balance
  ELSE IF choice = 3: DISPLAY balance
  ELSE IF choice = 4: EXIT LOOP
  ELSE: DISPLAY Invalid Choice
UNTIL exit
STOP
```

Python Source Code

```
balance = 5000

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")

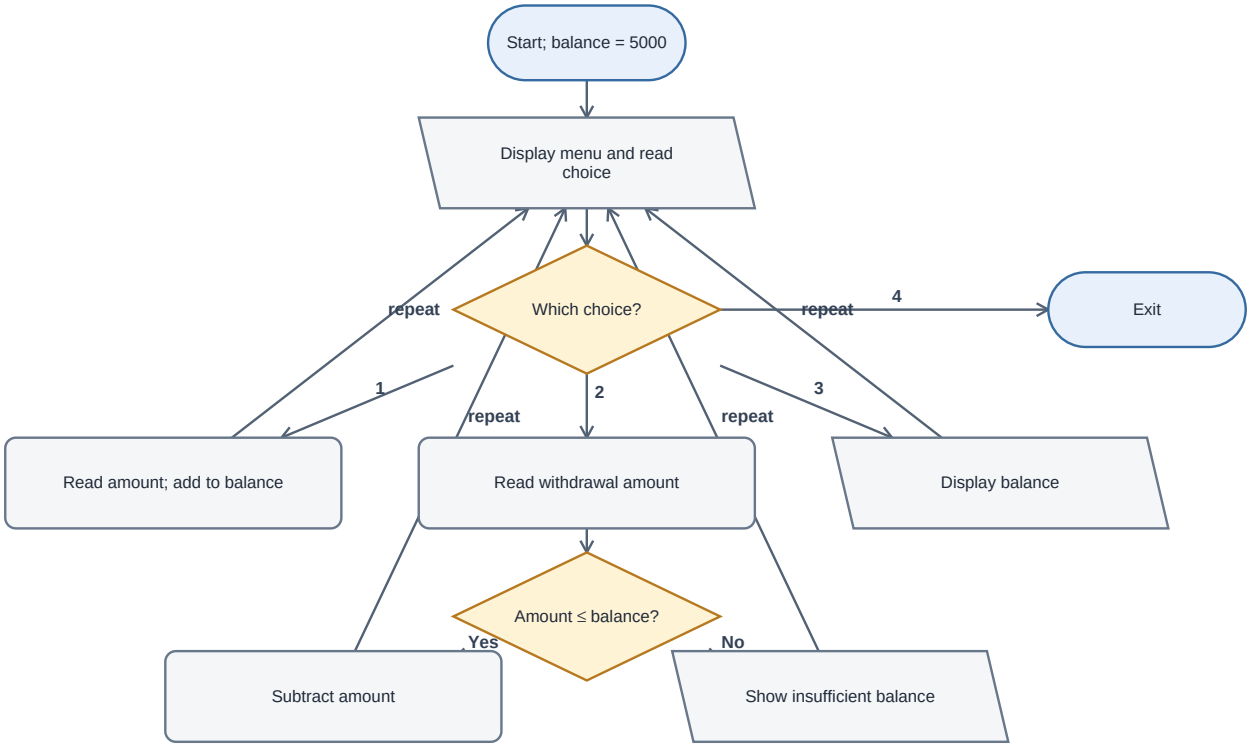
    choice = int(input("Enter choice: "))

    if choice == 1:
        amount = int(input("Enter amount: "))
        balance += amount
        print("Deposit successful")
    elif choice == 2:
        amount = int(input("Enter amount: "))
        if amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
```

```
        print("Insufficient Balance")
elif choice == 3:
    print("Balance:", balance)
elif choice == 4:
    print("Thank you")
    break
else:
    print("Invalid Choice")
```

Program 4: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 4 — Sample Run

Initial balance: 5000

1. Deposit   2. Withdraw   3. Check Balance   4. Exit
Enter choice: 3
Balance: 5000
Enter choice: 1
Enter amount: 1000
Deposit successful
Enter choice: 3
Balance: 6000
Enter choice: 2
Enter amount: 7000
Insufficient Balance
Enter choice: 4
Thank you
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 5: Product Billing with Discount

Aim: To calculate the total cost of products and apply a 10% discount when the bill is at least ■1,000.

Procedure

1. Read the number of products.
2. Set total to zero.
3. For each product, read its price and quantity.
4. Calculate price × quantity and add it to total.
5. If total is ■1,000 or more, deduct 10%.
6. Display the final bill.

Algorithm

1. Start.
2. Read the number of products.
3. Set total to zero.
4. For each product, read its price and quantity.
5. Calculate price × quantity and add it to total.
6. If total is ■1,000 or more, deduct 10%.
7. Display the final bill.
8. Stop.

Pseudocode

```
START
INPUT n
total ← 0
FOR each product from 1 to n
  INPUT price, quantity
  total ← total + price × quantity
END FOR
IF total ≥ 1000: total ← total − total × 0.10
DISPLAY total
STOP
```

Python Source Code

```
n = int(input("Enter number of products: "))
total = 0

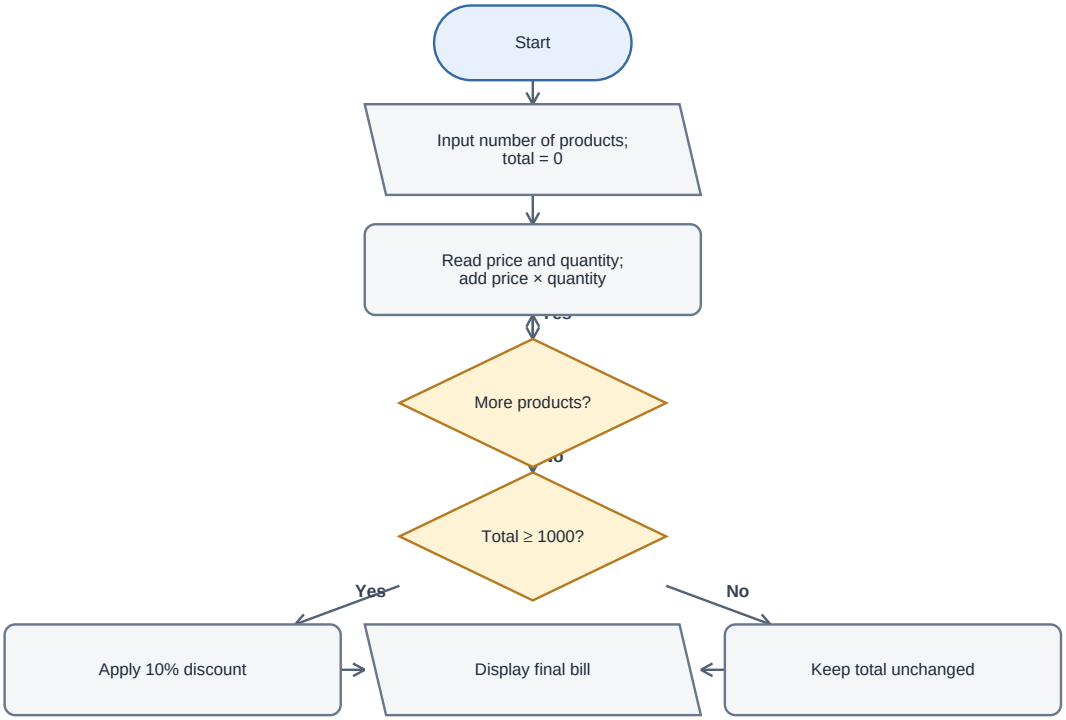
for i in range(n):
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))
    total += price * quantity

if total >= 1000:
    total = total - (total * 0.10)

print("Total Bill:", total)
```

Program 5: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 5 – Sample Run

Enter number of products: 2
Enter product price: 600
Enter quantity: 1
Enter product price: 250
Enter quantity: 2
Subtotal: 1100.0
Discount (10%): 110.0
Total Bill: 990.0
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 6: Class Attendance Eligibility

Aim: To calculate attendance percentage and determine eligibility based on a 75% requirement.

Procedure

1. Read total classes and attended classes.
2. Calculate attendance percentage as attended ÷ total × 100.
3. If the percentage is at least 75, display Eligible.
4. Otherwise, display Not Eligible.

Algorithm

1. Start.
2. Read total classes and attended classes.
3. Calculate attendance percentage as attended ÷ total × 100.
4. If the percentage is at least 75, display Eligible.
5. Otherwise, display Not Eligible.
6. Stop.

Pseudocode

```
START
INPUT total_classes, attended
percentage ← attended / total_classes × 100
IF percentage ≥ 75: DISPLAY Eligible
ELSE: DISPLAY Not Eligible
STOP
```

Python Source Code

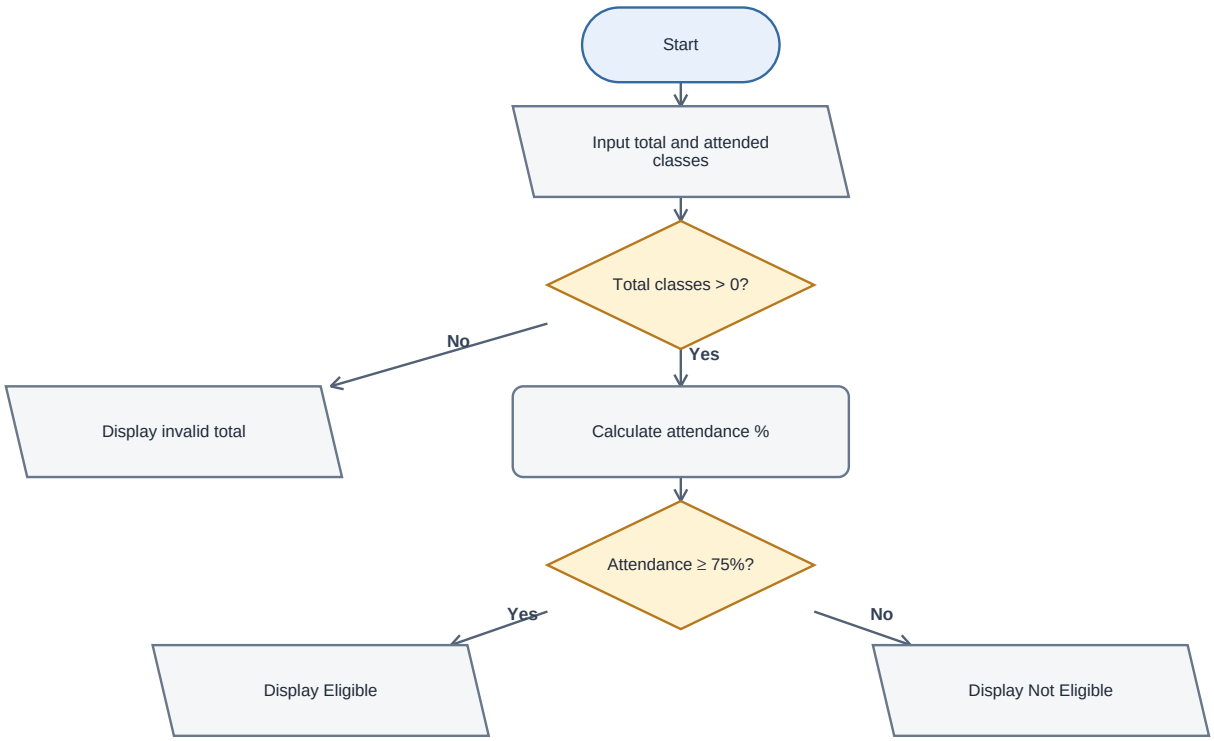
```
total_classes = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

if total_classes <= 0:
    print("Total classes must be greater than zero.")
else:
    percentage = (attended / total_classes) * 100
    print("Attendance Percentage:", percentage)

    if percentage >= 75:
        print("Eligible")
    else:
        print("Not Eligible")
```

Program 6: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 6 – Sample Run

Enter total classes: 40
Enter attended classes: 32
Attendance Percentage: 80.0
Eligible
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 7: Password Length Validation

Aim: To validate a password by checking whether it contains at least eight characters.

Procedure

1. Read a password from the user.
2. Find the number of characters using len().
3. If the length is at least 8, display Valid Password.
4. Otherwise, display Invalid Password.

Algorithm

1. Start.
2. Read a password from the user.
3. Find the number of characters using len().
4. If the length is at least 8, display Valid Password.
5. Otherwise, display Invalid Password.
6. Stop.

Pseudocode

```
START
INPUT password
IF length(password) ≥ 8
  DISPLAY Valid Password
ELSE
  DISPLAY Invalid Password
END IF
STOP
```

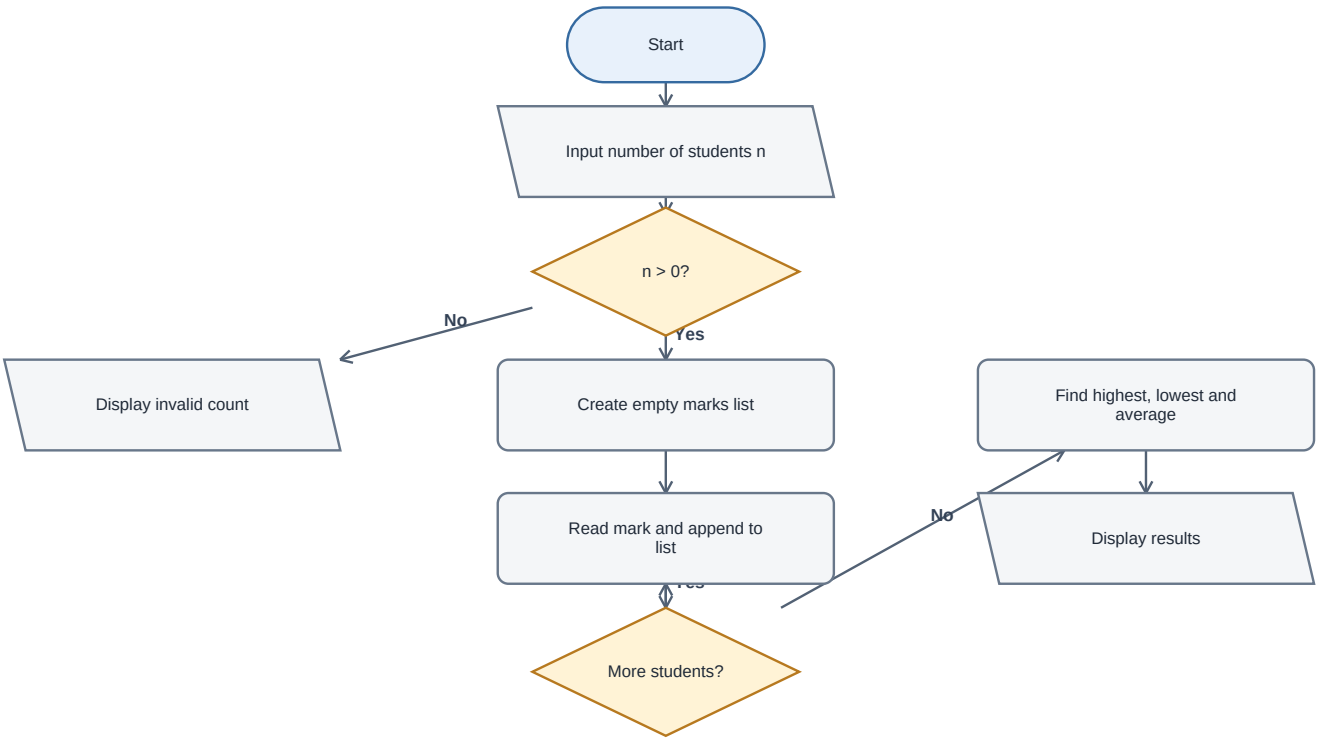
Python Source Code

```
password = input("Enter password: ")

if len(password) >= 8:
    print("Valid Password")
else:
    print("Invalid Password")
```

Program 7: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot



Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 8: Highest, Lowest and Average Marks

Aim: To accept marks of students and find the highest, lowest and average marks.

Procedure

1. Read the number of students.
2. Create an empty list to store marks.
3. Read each student's marks and append them to the list.
4. Use max() to find the highest mark and min() to find the lowest mark.
5. Use sum() ÷ number of students to calculate the average.
6. Display all three results.

Algorithm

1. Start.
2. Read the number of students.
3. Create an empty list to store marks.
4. Read each student's marks and append them to the list.
5. Use max() to find the highest mark and min() to find the lowest mark.
6. Use sum() ÷ number of students to calculate the average.
7. Display all three results.
8. Stop.

Pseudocode

```
START
INPUT n
IF n ≤ 0: DISPLAY invalid count and STOP
marks ← empty list
REPEAT n times: INPUT mark; APPEND mark to marks
highest ← MAX(marks)
lowest ← MIN(marks)
average ← SUM(marks) / n
DISPLAY highest, lowest, average
STOP
```

Python Source Code

```
marks = []

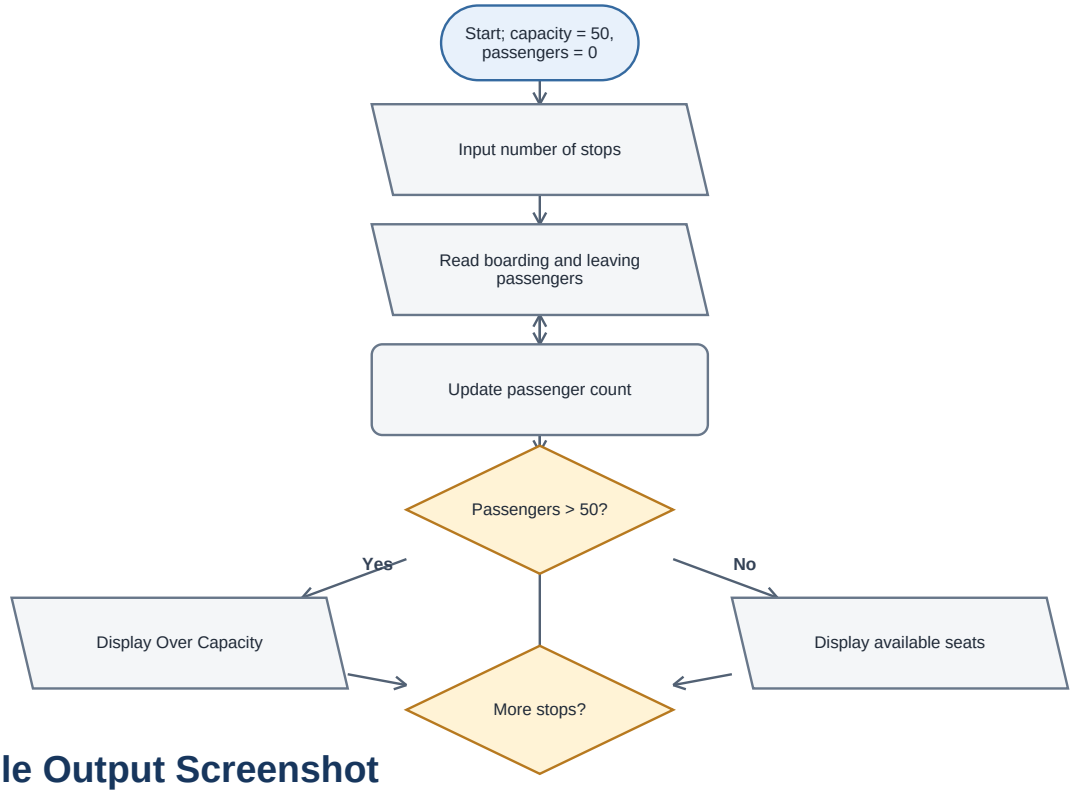
n = int(input("Enter number of students: "))

if n <= 0:
    print("Number of students must be greater than zero.")
else:
    for i in range(n):
        m = int(input("Enter marks: "))
        marks.append(m)

    print("Highest Marks:", max(marks))
    print("Lowest Marks:", min(marks))
    print("Average Marks:", sum(marks) / n)
```

Program 8: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 8 – Sample Run

Enter number of students: 4
Enter marks: 78
Enter marks: 92
Enter marks: 65
Enter marks: 85
Highest Marks: 92
Lowest Marks: 65
Average Marks: 80.0
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 9: Bus Passenger Capacity

Aim: To track passengers at each stop and report available seats or over-capacity status.

Procedure

1. Set bus capacity to 50 and initial passengers to zero.
2. Read the number of stops.
3. At each stop, read boarding and leaving passengers.
4. Update passengers as current passengers + boarding – leaving.
5. If passengers exceed capacity, display Over Capacity.
6. Otherwise, display the number of available seats.

Algorithm

1. Start.
2. Set bus capacity to 50 and initial passengers to zero.
3. Read the number of stops.
4. At each stop, read boarding and leaving passengers.
5. Update passengers as current passengers + boarding – leaving.
6. If passengers exceed capacity, display Over Capacity.
7. Otherwise, display the number of available seats.
8. Stop.

Pseudocode

```
START
capacity ← 50; passengers ← 0
INPUT n (number of stops)
FOR each stop from 1 to n
  INPUT boarding, leaving
  passengers ← passengers + boarding – leaving
  IF passengers > capacity: DISPLAY Over Capacity
  ELSE: DISPLAY capacity – passengers
END FOR
STOP
```

Python Source Code

```
capacity = 50
passengers = 0

n = int(input("Enter number of stops: "))

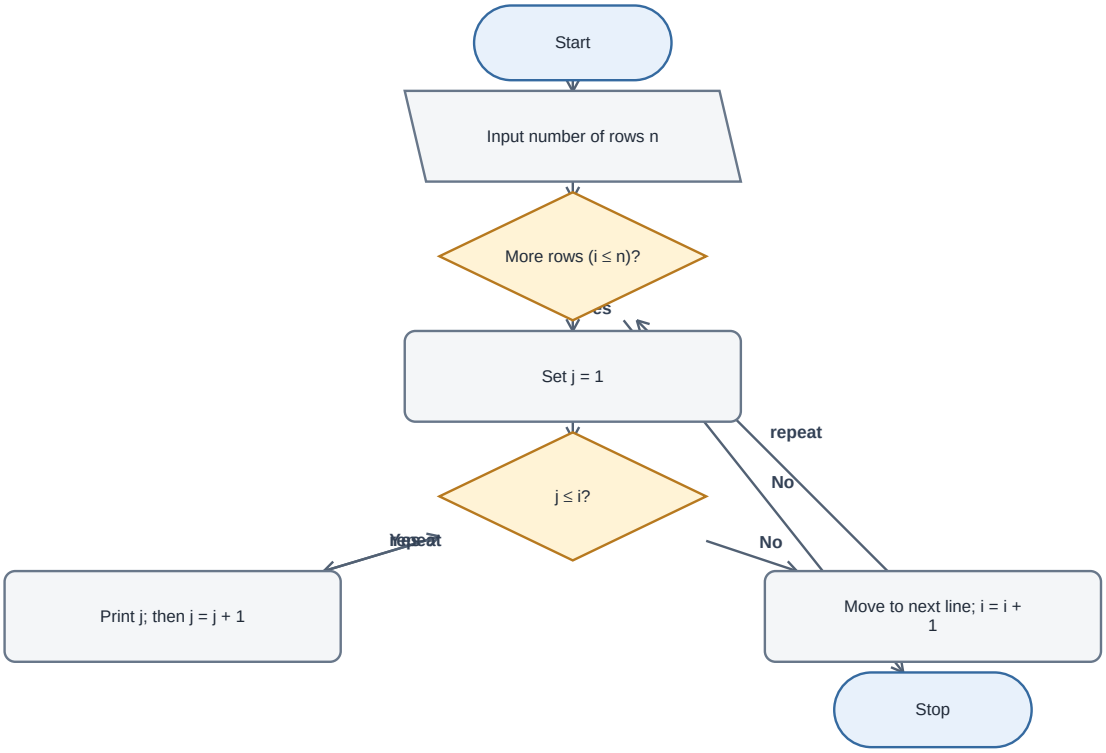
for i in range(n):
    boarding = int(input("Enter boarding passengers: "))
    leaving = int(input("Enter leaving passengers: "))

    passengers = passengers + boarding - leaving

    if passengers > capacity:
        print("Over Capacity")
    else:
        print("Available Seats:", capacity - passengers)
```

Program 9: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 9 – Sample Run

Enter number of stops: 3
Stop 1 - Enter boarding passengers: 20
Stop 1 - Enter leaving passengers: 0
Available Seats: 30
Stop 2 - Enter boarding passengers: 15
Stop 2 - Enter leaving passengers: 5
Available Seats: 20
Stop 3 - Enter boarding passengers: 25
Stop 3 - Enter leaving passengers: 3
Over Capacity
```

Result: The program was executed using the sample inputs shown and produced the displayed output.

Program 10: Number Pattern

Aim: To print a right-angled number pattern using nested for loops.

Procedure

1. Read the number of rows.
2. Run the outer loop from 1 to n.
3. For each row i, run the inner loop from 1 to i.
4. Print the inner-loop number on the same line with a space.
5. Move to the next line after each row.

Algorithm

1. Start.
2. Read the number of rows.
3. Run the outer loop from 1 to n.
4. For each row i, run the inner loop from 1 to i.
5. Print the inner-loop number on the same line with a space.
6. Move to the next line after each row.
7. Stop.

Pseudocode

```
START
INPUT n
FOR i ← 1 TO n
  FOR j ← 1 TO i
    DISPLAY j without newline
  END FOR
  MOVE to next line
END FOR
STOP
```

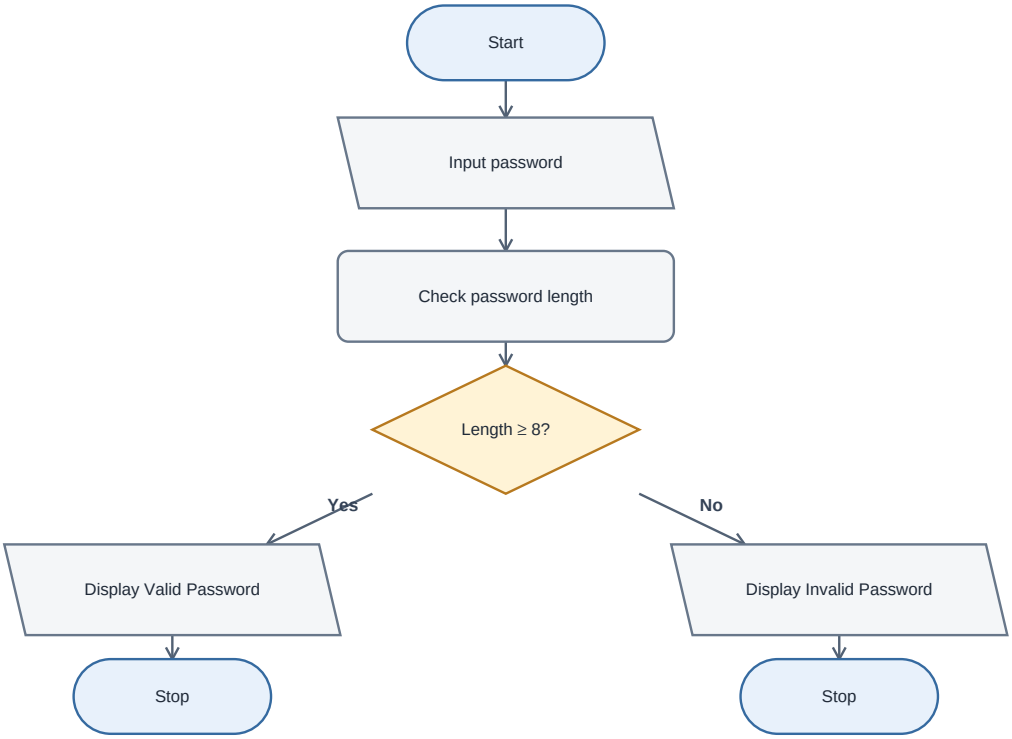
Python Source Code

```
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Program 10: Corrected Flowchart and Sample Output

Flowchart



Sample Output Screenshot

```
Program 10 – Sample Run

Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Result: The program was executed using the sample inputs shown and produced the displayed output.